

TP de SQL avec MySQL

Dr. Cheikh Ibrahima Fall DIOP

✉ : mamecheikhdio@gmail.com

☎ : (+221)7 $\left[\begin{smallmatrix} 7 \\ 6 \end{smallmatrix}\right]$ 543 01 00

1 Installation de MySQL

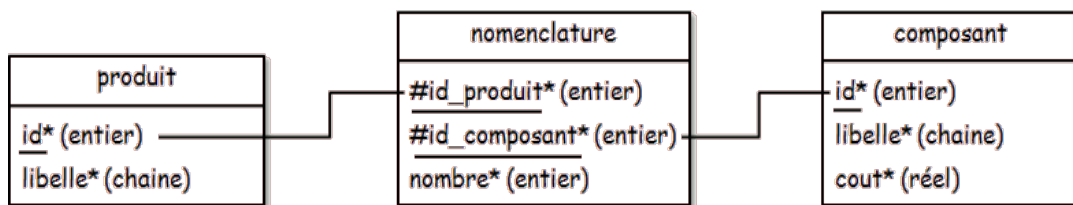
- ✓ Aller sur le site <https://dev.mysql.com/downloads/mysql/#downloads>
- ✓ Télécharger et installer (avec les options *serveur only* et *Developpement computer*) la version compatible son SE

2 Prise en mains de l'environnement

1. **Lancement**
 - ☐ Lancer MySQL Command Line Client
2. **Afficher l'aide de la console**
 - ☐ help
3. **Quitter la console**
 - ☐ quit ou exit

3 Requêtes d'interrogations simples

Soit le schéma relationnel de la base suivante concernant un atelier de fabrication :



Exécutez le fichier SQL pour créer et remplir les tables puis écrivez les requêtes pour répondre aux questions suivantes

1. Lister les libellés de tous les produits.
2. Récupérer le libellé du produit N 2.
3. Récupérer le libellé et le coût du composant N 3.
4. Lister les libellés des composants dont le coût est supérieur à 15 et inférieur ou égal à 45.
5. Récupérer le coût du composant le plus cher.

4 Encore plus loin avec les requêtes

4.1 Organiser des données identiques en groupes - GROUP BY et HAVING

La clause GROUP BY permet d'organiser des données identiques en groupes à l'aide de certaines fonctions. C'est-à-dire si une colonne particulière a les mêmes valeurs dans différentes lignes, elle organisera ces lignes dans un groupe.

La clause **GROUP BY** est utilisée avec l'instruction **SELECT**.

Dans la requête, la clause **GROUP BY** est placée après la clause **WHERE** et avant la clause **ORDER BY** si elle est utilisée.

Vous pouvez également utiliser certaines fonctions d'agrégation telles que **COUNT**, **SUM**, **MIN**, **MAX**, **AVG**, etc. sur la colonne groupée.

Syntaxe :

```
SELECT colonne1, colonne2, ... colonneN,  
fonction_agregation (nom_colonne)  
FROM tables  
[WHERE conditions]  
GROUP BY colonne1, colonne2, ... colonneN;
```

colonne1, colonne2, ... colonneN - spécifie les colonnes(ou expressions) qui ne sont pas encapsulées dans une fonction d'agrégation et doivent être incluses dans la clause **GROUP BY**.

fonction_agregation (nom_colonne) - Nom de la fonction d'agrégation utilisée, par exemple, **SUM()**, **AVG ()**,

WHERE conditions - C'est optionnel. Elle spécifie les conditions qui doivent être remplies pour que les enregistrements soient sélectionnés.

Soit les deux tables suivantes

Table - Employes

Id	Nom	Age	Salaire	Profession	Dep
1	Ismail	25	6000.00	Assistant	2
2	Mohamed	30	8000.40	Directeur	1
3	Fatima	29	6000.00	Directeur	3
4	Dounia	30	7000.00	Assistant	4
5	Omar	30	9000.00	Ingenieur	1
6	Mostafa	29	7500.00	Ingenieur	NULL

Table - Departement

Id_dep	Nom_dep
1	Informatique
2	RH
3	Vente
4	Strategies

Regroupement à l'aide d'une seule colonne Regroupement à l'aide d'une seule colonne signifie placer toutes les lignes ayant la même valeur que cette colonne particulière dans un groupe.

Pour mieux comprendre l'effet de la clause **GROUP BY**, exécutons une requête simple qui renvoie toutes les entrées Age de la table Employes.

```
SELECT Age FROM Employes;
```

```

+-----+
| Age |
+-----+
| 25 |
| 30 |
| 29 |
| 30 |
| 30 |
| 30 |
| 29 |
+-----+

```

Supposons que nous voulions obtenir les valeurs uniques pour Age. Nous pouvons utiliser une requête suivante

```
SELECT Age FROM Employes GROUP BY Age;
```

```

+-----+
| Age |
+-----+
| 25 |
| 29 |
| 30 |
+-----+

```

Notez que seuls trois résultats ont été renvoyés. C'est parce que nous n'avons que trois âges différents. La clause GROUP BY a regroupée tous les employés de la valeur Age="25" et n'a renvoyée qu'une seule ligne. La même chose avec les valeurs "29" et "30".

Groupe 1 - Age = 25

```

+-----+
| Age |
+-----+
| 25 |
+-----+

```

Groupe 2 - Age=29

```

+-----+
| Age |
+-----+
| 29 |
| 29 |
+-----+

```

Groupe 3 - Age = 30

```

+-----+
| Age |
+-----+
| 30 |
| 30 |
| 30 |
+-----+

```

Vous pouvez maintenant compter le nombre d'employés de chaque groupe à l'aide de la clause COUNT

```
SELECT Age, count(*) AS "Nombre d'employs" FROM Employes GROUP BY Age;
```

```

+-----+-----+
| Age | Nombre d'employés |
+-----+-----+
| 25 | 1 |
| 29 | 2 |
| 30 | 3 |
+-----+-----+

```

Exemple 1 : Compter le salaire moyen sur chaque groupe d'âge

```
SELECT Age, AVG(Salaire) AS "Salaire moyen" FROM Employes GROUP BY Age;
```

```

+-----+-----+
| Age | Salaire moyen |
+-----+-----+
| 25 | 6000.000000 |
| 29 | 6750.000000 |
| 30 | 8000.133333 |
+-----+-----+

```

Exemple 2 : Récupérez l'identifiant du département et le nombre d'employés dans chaque département.

```
SELECT Dep, COUNT(Dep) AS "Nombre d'employs" FROM Employes GROUP BY Dep;
```

```

+-----+-----+
| Dep | Nombre d'employés |
+-----+-----+
| NULL | 0 |
| 1 | 2 |
| 2 | 1 |
| 3 | 1 |
| 4 | 1 |
+-----+-----+

```

Exemple 3 : Récupérez le nom du département et le nombre d'employés dans chaque département.

```
SELECT D.Nom_dep, COUNT(E.Dep) AS "Nombre d'employs"
FROM Employes AS E INNER JOIN Departement AS D
```

```
ON E.Dep=D.Id_dep
GROUP BY D.Nom_dep;
```

Nom_dep	Nombre d'employés
Informatique	2
RH	1
Strategies	1
Vente	1

Regroupement sur plusieurs colonnes Regroupement par plusieurs colonnes, par exemple, GROUP BY colonne1, colonne2. Cela signifie placer toutes les lignes avec les mêmes valeurs des colonnes colonne1 et colonne2 dans un groupe.

Exemple 4 : Supposons maintenant que nous voulions regrouper tous les employés en fonction de leur âge et de leur département.

```
SELECT Age, Dep, COUNT(*) FROM Employes GROUP BY Age, Dep;
```

Age	Dep	COUNT(*)
25	2	1
29	NULL	1
29	3	1
30	1	2
30	4	1

Exemple 5 :

```
SELECT D.Nom_dep, E.Age , COUNT(E.Dep) AS "Nombre d'employs"
FROM Employes AS E INNER JOIN Departement AS D
ON E.Dep=D.Id_dep
GROUP BY D.Nom_dep, E.Age;
```

Nom_dep	Age	Nombre d'employés
Informatique	30	2
RH	25	1
Strategies	30	1
Vente	29	1

Clause HAVING Nous savons que la clause WHERE est utilisée pour imposer des conditions aux colonnes, mais que se passe-t-il si nous voulons imposer des conditions aux groupes ?

C'est ici que la clause HAVING entre en vigueur. Nous pouvons utiliser la clause HAVING pour poser des conditions afin de décider quel groupe fera partie de l'ensemble des résultats finaux. De plus, nous ne pouvons pas utiliser les fonctions d'agrégation telles que SUM(), COUNT(), etc. avec la clause WHERE. Nous devons donc utiliser la clause HAVING si nous voulons utiliser l'une de ces fonctions dans les conditions.

Syntaxe :

```
SELECT colonne1, colonne2, ... colonneN,  
fonction_agregation (nom_colonne)  
FROM tables  
[WHERE conditions]  
GROUP BY colonne1[, colonne2, ... colonneN]  
HAVING condition ;
```

Exemple 6 : La requête suivante récupère les noms de département et le salaire moyen de chaque département

```
SELECT D.Nom_dep, AVG(E.Salaire) AS "Salaire moyen"  
FROM Employes AS E INNER JOIN Departement AS D  
ON E.Dep=D.Id_dep  
GROUP BY D.Nom_dep;
```

Nom_dep	Salaire moyen
Informatique	8500.200000
RH	6000.000000
Strategies	7000.000000
Vente	6000.000000

Supposons maintenant que nous ne voulions montrer que les départements dont le salaire moyen est supérieur à 6000 ?

```
SELECT D.Nom_dep, AVG(E.Salaire) AS "Salaire moyen"  
FROM Employes AS E INNER JOIN Departement AS D  
ON E.Dep=D.Id_dep  
GROUP BY D.Nom_dep  
HAVING AVG(E.Salaire) > 6000;
```

Nom_dep	Salaire moyen
Informatique	8500.200000
Strategies	7000.000000

4.2 Les sous-requêtes en SQL

Une sous-requête, également appelée requête imbriquée ou sous-sélection, est une requête SELECT intégrée à la clause WHERE ou HAVING d'une autre requête SQL. Les données renvoyées par la sous-requête sont utilisées par l'instruction externe de la même manière qu'une valeur littérale serait utilisée.

Les sous-requêtes constituent un moyen simple et efficace de gérer les requêtes qui dépendent des résultats d'une autre requête. Elles sont presque identiques aux instructions SELECT normales, mais il existe peu de restrictions. Les plus importantes sont énumérées ci-dessous :

- ✓ Une sous-requête doit toujours apparaître entre parenthèses.
- ✓ Une sous-requête doit renvoyer une seule colonne. Cela signifie que vous ne pouvez pas utiliser SELECT * dans une sous-requête à moins que la table à laquelle vous faites référence ne comporte qu'une seule colonne. Vous pouvez utiliser une sous-requête qui renvoie plusieurs colonnes si le but est la comparaison de lignes.

- ✓ Vous ne pouvez utiliser que des sous-requêtes renvoyant plusieurs lignes avec des opérateurs de valeurs multiples, tels que l'opérateur IN ou NOT IN.
- ✓ Une clause ORDER BY ne peut pas être utilisée dans une sous-requête, bien que la requête principale puisse utiliser un ORDER BY. La clause GROUP BY peut être utilisée pour exécuter la même fonction que ORDER BY dans une sous-requête.
- ✓ Une sous-requête ne peut pas être une UNION. Une seule instruction SELECT est autorisée.

Les sous-requêtes sont le plus souvent utilisées avec l'instruction SELECT. Toutefois, vous pouvez également les utiliser dans une instruction INSERT, UPDATE ou DELETE ou dans une autre sous-requête.

Pendant cette section, nous allons considerer ces 4 tables Table - Departement

Id_dep	Nom_dep
1	Informatique
2	RH
3	Vente
4	Strategies

Table - Employes

Id	Nom	Age	Salaire	Profession	Dep
1	Ismail	25	6000.00	Assistant	2
2	Mohamed	30	8000.40	Directeur	1
3	Fatima	29	6000.00	Directeur	3
4	Dounia	30	7000.00	Assistant	4
5	Omar	30	9000.00	Ingenieur	1
6	Mostafa	29	7500.00	Ingenieur	3

Table - Projet

Id_p	Titre	Cout	Date_realisation
1	SIG Meknes	200000.00	2019-01-25
2	SI Vente	130000.00	2019-03-12
3	Blog RH	30000.00	2018-05-03

Table - Participer

```

+-----+-----+
| Emp | Proj |
+-----+-----+
| 1 | 1 |
| 2 | 1 |
| 2 | 3 |
| 3 | 1 |
| 6 | 2 |
| 6 | 3 |
+-----+-----+

```

Sous-requêtes avec l'instruction SELECT Les sous-requêtes sont le plus souvent utilisées avec l'instruction SELECT. La syntaxe de base est la suivante : Syntaxe :

```

SELECT nom_colonne [, nom_colonne ]
FROM table1 [, table2 ]
WHERE nom_colonne OPERATOR
      (SELECT nom_colonne [, nom_colonne ]
       FROM table1 [, table2 ]
       [WHERE])

```

Il existe principalement deux types de sous-requêtes :

1. Sous-requêtes indépendantes
2. Sous-requêtes corrélées

4.2.1 Sous-requêtes indépendantes

Dans les sous-requêtes indépendantes, l'exécution de la requête commence de la requête la plus interne vers la plus externe. L'exécution d'une requête interne est indépendante de la requête externe, mais le résultat de la requête interne est utilisé dans l'exécution de la requête externe. Divers opérateurs tels que IN, NOT IN, ANY, ALL, etc. sont utilisés pour l'écriture de sous-requêtes indépendantes.

Exemple 1 : Si nous voulons connaître les Ids des employés qui participent aux projets «Vente SI» ou «Blog RH», nous pouvons l'écrire à l'aide d'une sous-requête indépendante et d'un opérateur IN. A partir de la table Projet, nous pouvons trouver Id_p pour les projet «SI Vente» ou «Blog RH» et nous pouvons utiliser ces Id_p pour trouver Ids des employés à partir de la table Participer.

Nous pouvons le faire en deux étapes :

Etape 1 - Trouver Id_p pour les projet «SI Vente» ou «Blog RH»

```

SELECT Id_p FROM Projet WHERE Titre="Blog RH" OR Titre="Vente SI"

```

Etape 2 - utiliser les Id_p pour de la première étape pour trouver les Ids des employés à partir de la table Participer

```

SELECT DISTINCT Emp FROM Participer
WHERE Proj IN (SELECT Id_p FROM Projet WHERE Titre="Blog RH" OR Titre="Vente SI");

```

```

+-----+
| Emp |
+-----+
| 2 |
| 6 |
+-----+

```

Exemple 1 : Si nous voulons les noms de départements dans lesquels un employé a participé à un projet

Etape 1 - Trouver Dep des employés ayant participés dans un projet

```
SELECT DISTINCT E.Dep FROM Employes AS E INNER JOIN Participer AS P ON E.Id=P.Emp;
```

```
+-----+
| Dep   |
+-----+
|    2  |
|    1  |
|    3  |
+-----+
```

Etape 2 - Utiliser ces Ids pour trouver les noms des départements de la table Departement

```
SELECT Nom_dep FROM Departement
WHERE ID_dep IN (SELECT DISTINCT E.Dep FROM Employes AS E INNER JOIN Participer AS P ON
E.Id=P.Emp);
```

```
+-----+
| Nom_dep |
+-----+
| Informatique |
| RH       |
| Vente    |
+-----+
```

Exemple 1 : Pour trouver des employés qui ne participent à aucun projet

```
SELECT Nom FROM Employes
WHERE Id NOT IN (SELECT DISTINCT Emp FROM Participer);
```

```
+-----+
| Nom   |
+-----+
| Dounia |
| Omar  |
+-----+
```

4.2.2 Sous-requêtes corrélées

Une sous-requête corrélée est une sous-requête dont le résultat est différent selon les valeurs de la ligne de la requête externe pour laquelle la sous-requête est exécutée. Cela rend nécessaire d'exécuter la sous-requête pour chaque ligne extraite par la requête externe et ajoute au coût de performance de la requête.

Exemple 1 : Trouver l'employé avec le plus grand salaire dans son département

```
SELECT Nom FROM Employes AS e1
WHERE e1.Salaire=(SELECT MAX(e2.Salaire) FROM Employes AS e2 WHERE e1.Dep=e2.Dep);
```

```
+-----+
| Nom    |
+-----+
| Ismail |
| Dounia |
| Omar   |
| Mostafa|
+-----+
```

Dans la requête ci-dessus, la sous-requête doit être exécutée pour chaque ligne de la table `Employes`, car chaque employé peut appartenir à un service différent. Par conséquent, comparer son salaire au salaire maximum de son service signifie qu'il peut potentiellement être comparé à un nombre différent.

La requête doit exécuter la sous-requête pour chaque valeur distincte de `Dep`. Elle peut également trouver un moyen de mettre en cache les résultats de ces sous-requêtes et de les utiliser pour comparer les lignes d'employés suivantes avec le même `Dep`. Certaines implémentations `SQL` peuvent effectuer cette optimisation, d'autres non.

Si la sous-requête n'est pas corrélée, elle renverra logiquement le même résultat, quelles que soient les valeurs de la ligne de la requête externe. L'optimiseur `SQL` peut être conçu pour factoriser complètement la sous-requête, de sorte qu'elle n'ait à exécuter qu'une seule fois, quel que soit le nombre de lignes de la requête externe qui utilisent le résultat. C'est souvent une grande amélioration pour la performance.

4.2.3 Sous-requêtes vs jointures

Comparées aux jointures, les sous-requêtes sont simples à utiliser et à lire. Ils ne sont pas aussi compliqués que les jointures. Par conséquent, les débutants en `SQL` l'utilisent fréquemment.

Mais les sous-requêtes posent des problèmes de performances. L'utilisation d'une jointure au lieu d'une sous-requête peut parfois vous donner un gain de performances jusqu'à 500 fois. Si vous avez le choix, il est recommandé d'utiliser une jointure plutôt qu'une sous-requête.

Les sous-requêtes ne doivent être utilisées comme solution de secours que lorsque vous ne pouvez pas utiliser une opération `JOIN` pour atteindre les objectifs ci-dessus.

Sous-requêtes avec l'instruction `INSERT` La sous-requête `SQL` peut également être utilisée avec l'instruction `INSERT`. Dans l'instruction `INSERT`, les données renvoyées par la sous-requête sont utilisées pour être insérées dans une autre table.

Dans la sous-requête, les données sélectionnées peuvent être modifiées avec n'importe laquelle des fonctions de caractère et de date.

Syntaxe :

```
INSERT INTO nom_table [ (col1 [, col2 ]) ]
  SELECT [ *|col1 [, col2 ]
  FROM table1 [, table2 ]
  [ WHERE VALUE OPERATOR ]
```

Exemple 1 : Considérons une table `Employes_SAUVEGARDE` avec une structure similaire à la table `Employes`. Maintenant, pour copier la table `Employes` dans la table `Employes_SAUVEGARDE`, vous pouvez utiliser la syntaxe suivante.

```
INSERT INTO Employes_SAUVEGARDE
  SELECT * FROM Employes
```

Sous-requêtes avec l'instruction `UPDATE` La sous-requête peut être utilisée conjointement avec l'instruction `UPDATE`. Lorsqu'une sous-requête est utilisée avec l'instruction `UPDATE`, une ou plusieurs colonnes d'une table peuvent être mises à jour.

Syntaxe :

```
UPDATE table
  SET nom_colonne = nouvelle_valeur
```

```
[WHERE OPERATOR [VALUE]
(SELECT COLUMN_NAME
FROM TABLE_NAME)
[WHERE)]
```

Exemple 1 : En supposant que nous ayons une table Employes_SAUVEGARDE disponible qui est une sauvegarde de la table Employes. L'exemple suivant met à jour le salaire de 0,25 fois dans la table Employes pour tous les employés dont l'âge est supérieur ou égal à 30.

```
UPDATE Employes
SET Salaire = Salaire * 0.25
WHERE Age IN (SELECT Age FROM Employes_SAUVEGARDE
WHERE Age >= 30);
```

Table - Employes

Id	Nom	Age	Salaire	Profession	Dep
1	Ismail	25	6000.00	Assistant	2
2	Mohamed	30	2000.10	Directeur	1
3	Fatima	29	6000.00	Directeur	3
4	Dounia	30	1750.00	Assistant	4
5	Omar	30	2250.00	Ingenieur	1
6	Mostafa	29	7500.00	Ingenieur	3

Sous-requêtes avec l'instruction DELETE La sous-requête peut être utilisée en conjonction avec l'instruction DELETE, exactement comme toutes les instructions mentionnées ci-dessus.

Exemple 1 : Supposons que nous ayons une table Employes_SAUVEGARDE disponible qui est une sauvegarde de la table Employes. L'exemple donné supprime les enregistrements de la table Employes pour tous les employés dont l'âge est inférieur ou égal à 29.

```
DELETE FROM Employes
WHERE Age IN (SELECT Age FROM Employes_SAUVEGARDE
WHERE Age <= 29 );
```

Table - Employes

Id	Nom	Age	Salaire	Profession	Dep
2	Mohamed	30	2000.10	Directeur	1
4	Dounia	30	1750.00	Assistant	4
5	Omar	30	2250.00	Ingenieur	1

4.3 Combinez les résultats de deux ou plusieurs instructions SELECT - UNION

La clause/opérateur SQL UNION est utilisée pour combiner les résultats de deux ou plusieurs instructions SELECT sans renvoyer les lignes en double.

Pour utiliser cette clause UNION, chaque instruction SELECT doit avoir :

1. Le même nombre de colonnes sélectionnées
2. Le même type de données et
3. Les avoir dans le même ordre

Mais ils n'ont pas besoin d'être dans la même longueur.

La clause UNION produit des valeurs distinctes dans le jeu de résultats. Pour extraire les valeurs en double, UNION ALL doit être utilisée à la place de UNION.

Syntaxe - UNION

```
SELECT colonne1 [, colonne2 ]  
FROM table1 [, table2 ]  
[WHERE condition]
```

UNION

```
SELECT colonne1 [, colonne2 ]  
FROM table1 [, table2 ]  
[WHERE condition]
```

Syntaxe - UNION ALL

```
SELECT colonne1 [, colonne2 ]  
FROM table1 [, table2 ]  
[WHERE condition]
```

UNION ALL

```
SELECT colonne1 [, colonne2 ]  
FROM table1 [, table2 ]  
[WHERE condition]
```

Ici, la condition donnée peut être toute expression donnée en fonction de vos besoins.

Pendant ce section, nous allons travailler sur ces 2 tables Table - Employes

Id	Nom	Age	Salaire	Profession	Dep
1	Ismail	25	6000.00	Technicien	2
2	Mohamed	30	2000.10	Directeur	1
3	Fatima	29	8000.00	Assistant	2
4	Dounia	30	1750.00	Assistant	4
5	Omar	30	2250.00	Ingenieur	1
6	Mostafa	32	7500.00	Formateur	1

Table - Conges

Id_c	Date_debut	Date_fin	Raison	Emp
1	2019-02-01	2019-02-15	Voyage	1
2	2019-05-20	2019-06-03	Maladie d'enfant	3
3	2019-09-01	2019-09-15	Rentrée scolaire	4



Les noms de colonne dans les deux instructions SELECT peuvent être différents mais le type de données doit être identique. Et dans le résultat, le nom de la colonne utilisée dans la première instruction SELECT apparaîtra.

Exemples Exemple 1 : Pour récupérer des identifiants distincts d'employés à partir des tables Em-

ployee et Conges.

```
SELECT Id From Employes
UNION
SELECT Emp FROM Conges;
```

```
+-----+
| Id |
+-----+
| 1 |
| 2 |
| 3 |
| 4 |
| 5 |
| 6 |
+-----+
```

Exemple 2 : Pour récupérer des identifiants distincts d'employés de la table Employee et de la table Conges, y compris les valeurs en double.

```
SELECT Id From Employes
UNION ALL
SELECT Emp FROM Conges;
```

```
+-----+
| Id |
+-----+
| 1 |
| 2 |
| 3 |
| 4 |
| 5 |
| 6 |
| 1 |
| 3 |
| 4 |
+-----+
```

Exemple 3 : Pour récupérer Id et le Nom de la table Employes où Id est supérieur à 3, et Emp, Raison de la table Conges où Emp est inférieur à 3, en incluant les valeurs en double et en triant les données par Id.

```
SELECT Id, Nom From Employes WHERE Id > 3
UNION ALL
SELECT Emp, Raison FROM Conges WHERE Emp < 3
ORDER BY Id;
```

```

+-----+-----+
| Id | Nom      |
+-----+-----+
| 1 | Voyage   |
| 4 | Dounia   |
| 5 | Omar     |
| 6 | Mostafa  |
+-----+-----+

```

Exemple 4 : Utilisez LEFT JOIN et RIGHT JOIN dans la clause UNION

```

SELECT E.Id, E.Nom FROM Employes as E LEFT JOIN Conges as C ON E.Id=C.Emp
UNION
SELECT E.Id, E.Nom FROM Employes as E RIGHT JOIN Conges as C ON E.Id=C.Emp;

```

```

+-----+-----+
| Id | Nom      |
+-----+-----+
| 1 | Ismail   |
| 3 | Fatima   |
| 4 | Dounia   |
| 2 | Mohamed  |
| 5 | Omar     |
| 6 | Mostafa  |
+-----+-----+

```

4.4 Les fonctions SQL de manipulation de date

En SQL, les dates sont compliquées pour les débutants, car lors de l'utilisation d'une base de données, le format de la date dans le tableau doit être appariée avec la date d'entrée pour pouvoir fonctionner.

Dans divers scénarios au lieu de la Date, DateTime(l'heure est également impliquée avec la date) est utilisé.

Dans cette section, nous allons parler de différentes fonctions de date par défaut dans MySQL. Nous utiliserons la table échantillon - Conges suivante :

```

+-----+-----+-----+-----+-----+
| Id_c | Date_debut | Date_fin | Raison          | Emp |
+-----+-----+-----+-----+-----+
| 1 | 2019-02-01 | 2019-02-15 | Voyage          | 1 |
| 2 | 2019-05-20 | 2019-06-03 | Maladie d'enfant | 3 |
| 3 | 2019-09-01 | 2019-09-15 | Rentrée scolaire | 4 |
| 4 | 2019-09-24 | 2019-09-29 | Voyage          | 2 |
| 5 | 2019-09-19 | 2019-09-24 | Soutenance      | 5 |
+-----+-----+-----+-----+-----+

```

4.4.1 NOW()

Renvoie la date et l'heure actuelles sous forme de valeur au format 'AAAA-MM-JJ HH : MM : SS' ou YYYYMMDDHHMMSS, selon que la fonction est utilisée dans un contexte de chaîne ou numérique. La valeur est exprimée dans le fuseau horaire actuel.

Exemple :

```

SELECT NOW();

```

```
+-----+
| NOW() |
+-----+
| 2023-11-18 00:42:49 |
+-----+
```

4.4.2 CURDATE()

Renvoie la date actuelle sous forme de valeur au format 'AAAA-MM-JJ' ou AAAAMMMJJ, selon que la fonction est utilisée dans un contexte de chaîne ou numérique.

Exemple :

```
SELECT CURDATE();
```

```
+-----+
| CURDATE() |
+-----+
| 2023-11-18 |
+-----+
```

4.4.3 CURTIME()

Renvoie l'heure actuelle sous forme de valeur au format 'HH : MM : SS' ou HHMMSS, selon que la fonction est utilisée dans un contexte de chaîne ou numérique. La valeur est exprimée dans le fuseau horaire actuel.

Exemple :

```
SELECT CURTIME();
```

```
+-----+
| CURTIME() |
+-----+
| 00:45:00 |
+-----+
```

4.4.4 DATE(date)

Extrait la partie date de la date ou de l'expression DateTime.

Exemple 1 :

```
SELECT DATE('2023-11-18 00:45:03');
```

```
+-----+
| DATE('2023-11-18 00:45:03') |
+-----+
| 2023-11-18 |
+-----+
```

Exemple 2 :

```
SELECT DATE(Date_debut) FROM Conges;
```

```

+-----+
| DATE(Date_debut) |
+-----+
| 2019-02-01      |
| 2019-05-20      |
| 2019-09-01      |
| 2019-09-24      |
| 2019-09-19      |
+-----+

```

4.4.5 EXTRACT(unit FROM date)

Renvoie une seule partie d'une date/heure. Plusieurs unités peuvent être envisagées, mais seules certaines sont utilisées, telles que : MICROSECOND, SECOND, MINUTE, HOUR, DAY, WEEK, MONTH, QUARTER, YEAR, etc.

Exemple :

```

SELECT EXTRACT(DAY FROM Date_debut) as "Jour", EXTRACT(MONTH FROM Date_debut) as "Mois",
EXTRACT(YEAR FROM Date_debut) as "Anne"
FROM Conges;

```

```

+-----+-----+-----+
| Jour | Mois | Année |
+-----+-----+-----+
| 1    | 2    | 2019  |
| 20   | 5    | 2019  |
| 1    | 9    | 2019  |
| 24   | 9    | 2019  |
| 19   | 9    | 2019  |
+-----+-----+-----+

```

4.4.6 DATE_ADD(date, INTERVAL expr type)

Ajoute un intervalle de temps spécifié à une date. type peut être l'un des types suivants : MICROSECOND, SECOND, MINUTE, HOUR, DAY, WEEK, MONTH, QUARTER, YEAR, etc.

Exemple 1 : Ajouter un an à chaque Date_debut

```

SELECT DATE_ADD(Date_debut, INTERVAL 1 YEAR) FROM Conges;

```

```

+-----+
| DATE_ADD(Date_debut, INTERVAL 1 YEAR) |
+-----+
| 2020-02-01 |
| 2020-05-20 |
| 2020-09-01 |
| 2020-09-24 |
| 2020-09-19 |
+-----+

```

Exemple 2 : Ajouter 3 mois à chaque Date_debut

```

SELECT DATE_ADD(Date_debut, INTERVAL 3 MONTH) FROM Conges;

```

```

+-----+
| DATE_ADD(Date_debut, INTERVAL 3 MONTH) |
+-----+
| 2019-05-01 |
| 2019-08-20 |
| 2019-12-01 |
| 2019-12-24 |
| 2019-12-19 |
+-----+

```

4.4.7 DATE_SUB(date, INTERVAL expr type)

Soustrait un intervalle de temps spécifié à une date. type peut être l'un des types suivants : MICROSECOND, SECOND, MINUTE, HOUR, DAY, WEEK, MONTH, QUARTER, YEAR, etc.

Exemple : Soustrait 3 mois de Date_Debut

```
SELECT DATE_SUB(Date_debut, INTERVAL 3 MONTH) FROM Conges;
```

```

+-----+
| DATE_SUB(Date_debut, INTERVAL 3 MONTH) |
+-----+
| 2018-11-01 |
| 2019-02-20 |
| 2019-06-01 |
| 2019-06-24 |
| 2019-06-19 |
+-----+

```

4.4.8 DATEDIFF(date1, date2);

Renvoie date1-date2 sous forme de valeur en jours d'une date à l'autre. date1 et date2 sont des expressions de date ou de date et heure. Seules les parties de date des valeurs sont utilisées dans le calcul.

Exemple : La requête suivante indiquera le nombre de jours passés de Date_debut jusqu'à maintenant

```
SELECT DATEDIFF(NOW(), Date_debut) AS "Nombre jours passs" FROM Conges;
```

```

+-----+
| Nombre jours passés |
+-----+
| 235 |
| 127 |
| 23 |
| 0 |
| 5 |
+-----+

```

4.4.9 YEAR(date), MONTH(date) et DAY(date)

Utilisée pour extraire l'année, le mois et le jour respectivement

Exemple :

```
SELECT YEAR(Date_debut), MONTH(Date_debut), DAY(Date_debut) FROM Conges;
```

YEAR(Date_debut)	MONTH(Date_debut)	DAY(Date_debut)
2019	2	1
2019	5	20
2019	9	1
2019	9	24
2019	9	19

5

Requêtes d'interrogations complexes

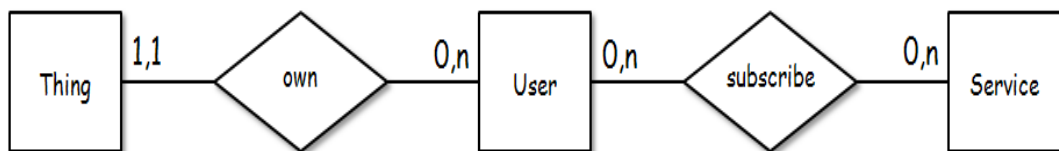
5.1 Requêtes sur la base `platform_iot`

Le contexte du cas traité dans ce premier exercice est l'internet des objets. Dans ce contexte, nous souhaitons modéliser une plateforme de gestion de services. Des utilisateurs, qui possèdent des objets connectés, souscrivent à des services. Lorsque les objets envoient des données, elles sont transmises aux services souscrits par leur propriétaire. Pour savoir à qui les objets connectés appartiennent et quels sont les services souscrits par leur propriétaire, nous utilisons une base de données.

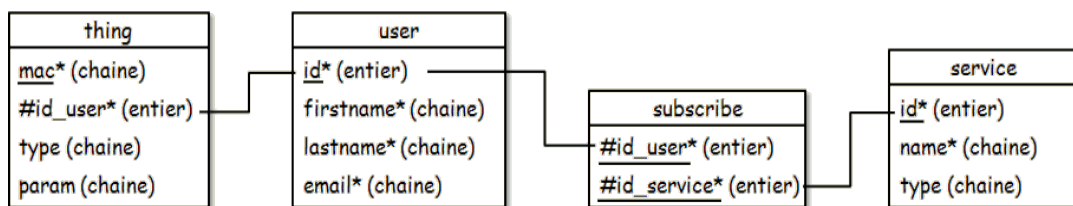
Les cardinalités entre les entités sont les suivantes :

- Un objet appartient à un et un seul utilisateur.
- Un utilisateur peut posséder plusieurs objets.
- Un utilisateur peut souscrire à plusieurs services.
- Un service peut être souscrit par plusieurs utilisateurs.

Le schéma Entité-Association de cette base de données est le suivant :



La traduction en schéma relationnel donne les quatre tables suivantes :



- ✓ Chaque objet connecté (décrit par la table `thing`) est identifié par son adresse réseau physique unique (adresse MAC) sur 17 caractères. Le champ `type` permet d'indiquer si l'objet est particulier (sinon, le champ n'est pas spécifié). Le champ `param` permet de fournir des paramètres optionnels concernant l'objet.
- ✓ Le champ `type` de la table `service` permet d'indiquer si le service est particulier (sinon, le champ n'est pas spécifié).

Créez la base puis exécutez le script SQL pour créer et peupler les tables et écrivez les requêtes pour répondre aux questions suivantes :

1. Quel est l'adresse email de l'utilisateur qui possède l'objet d'adresse MAC `f0:de:f1:39:7f:17` ?
2. Quels sont les adresses MAC des objets appartenant à l'utilisateur dont l'adresse email est `m.holzarte@company.fr` ?

3. Affichez les noms et prénoms des utilisateurs avec les noms des services auxquels ils sont abonnés.
4. Combien de services sont de type smarthome ?
5. Afficher les id des propriétaires d'objets avec le nombre d'objets qu'ils possèdent.
6. Afficher les noms et prénoms des propriétaires d'objets avec le nombre d'objets qu'ils possèdent.
7. Afficher les noms et prénoms des propriétaires de (strictement) plus de 1 objet.

5.2 Retour sur la base de l'atelier de fabrication

Écrivez les requêtes pour répondre aux questions suivantes :

1. Quels sont les libellés des produits qui utilisent des vis ?
2. Quels sont les libellés des produits qui utilisent huit vis ?
3. Quels sont les libellés des composants d'un siège ?
4. Afficher le libellé des composants avec le nombre de produits dans lesquels ils sont utilisés.
5. Afficher le libellé des composants qui sont utilisés dans 1 seul produit.
6. Afficher le libellé des composants d'un siège, avec la quantité utilisée, le coût unitaire et le coût final.
7. Afficher le coût total d'un siège.